

Module 11: Generics

Generics moved a whole class of failure from runtime to compile time. Before Java 5, every collection held `Object` and every read needed a cast you simply had to get right.

The feature is easy to use and easy to misread, because one design decision made in 2004 explains nearly every limitation you will hit. That decision is erasure, and this module leads with it.

What You'll Learn

- Generic classes and methods, and why bounds let them do anything useful
- Erasure, what it forbids, and why it was chosen anyway
- Raw types, heap pollution, and where the resulting crash actually lands
- Wildcards through PECS
- Why generics are invariant and arrays are not, and which choice was the mistake

Coming From Another Language

	Python	Java
Type parameters	hints, unenforced	enforced at compile time
At runtime	available via <code>__orig_class__</code>	erased, nothing survives
Variance	annotated on the TypeVar	wildcards at the use site
Primitives	no distinction	<code>List<int></code> is illegal

The erasure row is the one that will surprise you. In Java the type argument genuinely does not exist at runtime, and a surprising number of limitations follow from that one fact.

The Lesson

The basics

```
class Box<T> {  
    private final T value;  
    T get() { return value; }  
}  
  
static <T> T first(List<T> items) { return items.get(0); }
```

For a generic method the angle brackets come **before** the return type, and the type is inferred from the arguments.

Bounded type parameters are what let a generic method do more than shuffle values around:

```
static <T extends Comparable<T>> T max(List<T> items)
```

Without the bound, `T` is treated as `Object` and `compareTo` does not exist. Multiple bounds use `&`, with the class first: `<T extends Number & Comparable<T>>`.

Conventional names are `T` type, `E` element, `K` key, `V` value, `R` result. They are only conventions.

Erasure

From `Erasure.java`:

```
List<String> class: java.util.ArrayList
List<Integer> class: java.util.ArrayList
same class: true
```

The type argument left no trace. **Why it works this way:** generics arrived fifteen years after the JVM. Erasure meant generic and pre-generic code could call each other and run on an unchanged JVM, so existing libraries did not need rewriting. It was a deliberate trade, compatibility then for these limits forever.

What erasure forbids:

Forbidden	Because
<code>x instanceof List<String></code>	no such type exists at runtime
<code>new T[]</code> , <code>new T()</code>	nothing to allocate
<code>show(List<String>)</code> and <code>show(List<Integer>)</code>	same erasure, they collide
<code>List<int></code>	primitives are not objects
<code>static T shared;</code> inside <code>Box<T></code>	one static field per class, not per type argument

The standard workaround when you genuinely need the type at runtime is to pass a `Class<T>`, which is why so many library methods take one.

Raw types and heap pollution

`HeapPollution.java` compiles with a warning and then fails:

```
List<String> words = new ArrayList<>();
List raw = words;    // raw type, legal for backward compatibility
raw.add(42);         // unchecked warning, allowed
```

```
the list now contains: [legitimate, 42]
reading element 0 is fine: legitimate
reading element 1 will not be:
Exception in thread "main" java.lang.ClassCastException:
    class java.lang.Integer cannot be cast to class java.lang.String
```

Look at where the crash lands. The line that broke the invariant was `raw.add(42)`. The failure is at the `get`, because that is where the compiler inserted the cast the declared type promised. In real code those two lines are often in different classes written by different people, and that distance is what makes heap pollution hard to debug.

Never use a raw type in new code. Unchecked warnings mean the compiler has stopped being able to guarantee anything. When you must suppress one, put `@SuppressWarnings` on the narrowest possible scope and say why it is safe.

Wildcards: PECS

Producer Extends, Consumer Super.

Generics are **invariant**: a `List<Integer>` is not a `List<Number>`, even though an `Integer` is a `Number`. Without wildcards you would need one overload per element type.

```
// PRODUCER: reads from the list, so any subtype of Number will do
static double sum(List<? extends Number> numbers)

// CONSUMER: writes into the list, so any supertype of Integer will do
static void addAll(List<? super Integer> destination, List<Integer> source)
```

Each direction costs you the other:

- `? extends` is **read-only**. The list might be a `List<Double>`, so adding an `Integer` would corrupt it. Only `null` may be added.
- `? super` **reads as `Object`**. The list might be a `List<Object>`, so that is all the compiler can promise.

`List<?>` is the unbounded form: read `Object`s, check the size, add nothing. It differs from the raw `List` in the way that matters, since `List<?>` is type-safe and enforced while the raw type simply turns checking off.

Invariance was the right call

Arrays chose the other answer. From [Wildcards.java](#):

```
Object[] array = new String[2]; // compiles: arrays are covariant
array[0] = 42;                 // throws at runtime
```

```
ArrayStoreException: java.lang.Integer
```

Every array write carries a runtime type check because of that decision. Generics refuse the equivalent at compile time, so no check is needed and the failure cannot reach production.

Run It

```
java modules/11-generics/examples/Generics.java
java modules/11-generics/examples/Erasure.java
java modules/11-generics/examples/Wildcards.java

# These two fail on purpose, one at runtime and one at compile time.
java modules/11-generics/examples/HeapPollution.java
java modules/11-generics/examples/CannotCreateGenericArray.java

./scripts/verify-examples.sh modules/11-generics
```

Common Mistakes

Using a raw type to make a warning go away. You have turned off the checking the feature exists to provide.

`@SuppressWarnings` **on a whole class or method.** It hides every future mistake in that scope too. Put it on the single declaration that needs it.

Trying `new T[size]`. Use a `List`, or allocate `Object[]` and cast while keeping the array private to the class.

Writing `List<? extends Number>` for a parameter you need to add to. You cannot. If you both read and write, use a plain `List<T>`.

Wildcards on a return type. It forces every caller to deal with the wildcard. Return the concrete type.

Reaching for `Object` where a type parameter would do. Every caller then needs a cast, which is exactly the pre-2004 situation.

Key Takeaways

- **Generics are checked at compile time and erased at runtime.** Nearly every limitation follows from that.
- **Bounds are what let generic code do anything** beyond moving values around.
- **Raw types defeat the whole feature**, and the resulting crash appears far from the line that caused it.
- **PECS:** producer `extends` for reading, consumer `super` for writing.
- `? extends` **cannot be written to;** `? super` **reads as `Object`.**

- **Generics are invariant and arrays are covariant.** The array choice was the mistake, and `ArrayStoreException` is the bill.

Homework

[homework/README.md](#)

Build a generic container, hit three erasure limits, and cause heap pollution on purpose. Reference solution in [solutions/11-generics/](#).